

Git & GitHub

Maîtriser le contrôle de version moderne

Git	GitHub	Branches	Merge
-----	--------	----------	-------

Collaboration	CI/CD
---------------	-------

Parcours de formation — Développement d'Applications Mobiles

Djiguitech · Social Seller · Tous niveaux

■ Compétences visées

- ✓ Expliquer le fonctionnement interne de Git (objets, DAG, index)
- ✓ Utiliser les commandes fondamentales dans un workflow quotidien
- ✓ Concevoir une stratégie de branches adaptée à un projet en équipe
- ✓ Résoudre des conflits de fusion en comprenant leur origine
- ✓ Configurer des GitHub Actions pour automatiser tests et déploiement

Table des matières

Chapitre 1 — L'histoire et la philosophie du contrôle de version

- 1.1 Avant Git — les solutions centralisées
- 1.2 La naissance de Git
- 1.3 Décentralisation et intégrité

Chapitre 2 — Architecture interne de Git

- 2.1 Les quatre types d'objets
- 2.2 Le DAG — graphe acyclique dirigé
- 2.3 L'index (staging area)
- 2.4 Les références

Chapitre 3 — Workflow quotidien

- 3.1 Initialiser et cloner
- 3.2 Le cycle add / commit
- 3.3 Inspecter l'historique
- 3.4 Annuler et corriger

Chapitre 4 — Branches — travailler en parallèle

- 4.1 Qu'est-ce qu'une branche ?
- 4.2 Merge vs Rebase
- 4.3 Résolution de conflits
- 4.4 Cherry-pick et stash

Chapitre 5 — Collaboration avec GitHub

- 5.1 Remote, push, pull, fetch
- 5.2 Pull Requests et code review
- 5.3 Gitflow et trunk-based development
- 5.4 GitHub Actions

L'histoire et la philosophie du contrôle de version

1.1 Avant Git — les solutions centralisées

Avant que Linus Torvalds ne crée Git en 2005, les équipes de développement utilisaient des systèmes centralisés comme CVS (1990) ou Subversion (2000). Dans ce modèle, un seul serveur central détient l'historique complet du projet. Chaque développeur possède uniquement la dernière version des fichiers sur sa machine. Cette architecture présente une faiblesse majeure : le serveur central est un point de défaillance unique. Si le serveur tombe, personne ne peut commiter, consulter l'historique ni créer de branches. Travailler hors-ligne est impossible.

La performance est une autre limite critique. Chaque opération — créer une branche, consulter l'historique d'un fichier, comparer deux versions — nécessite un aller-retour réseau vers le serveur central. Sur de grands projets comme le noyau Linux (des millions de lignes de code, des centaines de contributeurs simultanés), ce modèle devient un goulot d'étranglement insoutenable.

1.2 La naissance de Git

En 2002, la communauté du noyau Linux utilise BitKeeper, un système propriétaire décentralisé. En 2005, suite à un désaccord de licence, l'accès gratuit est révoqué. Linus Torvalds décide de créer son propre système en posant des exigences non-négociables : vitesse, conception simple, support fort du développement non-linéaire (milliers de branches parallèles), distribution complète (pas de serveur central obligatoire), et capacité à gérer des projets de la taille du noyau Linux.

En quelques semaines, les premières versions de Git voient le jour. Le noyau Linux migre vers Git le 16 juin 2005. Aujourd'hui, Git est utilisé par plus de 90 % des développeurs professionnels dans le monde et est devenu la norme de facto pour le contrôle de version.

Définition — Système de contrôle de version (VCS)

Un VCS enregistre l'historique complet des modifications apportées à un ensemble de fichiers. Il permet de revenir à n'importe quel état passé, de comparer des versions, de travailler en parallèle sur des fonctionnalités distinctes, et de fusionner des travaux provenant de sources multiples.

1.3 Décentralisation et intégrité

Git est un système distribué (DVCS). Chaque clone d'un dépôt contient l'intégralité de l'historique — pas uniquement un pointeur vers un serveur. Cette propriété a des conséquences profondes : toutes les opérations sont locales et donc quasi-instantanées ; travailler sans connexion réseau est parfaitement possible ; la perte du serveur 'principal' n'entraîne aucune perte de données si un clone existe quelque

part.

L'intégrité est garantie par le hachage cryptographique SHA-1 (et progressivement SHA-256). Chaque objet Git est identifié par l'empreinte SHA-1 de son contenu. Si un seul octet d'un fichier est modifié — accidentellement ou malicieusement — son identifiant change. Il est donc impossible de modifier l'historique sans que Git ne le détecte immédiatement.

■ Note

Le passage de SHA-1 à SHA-256 est en cours dans les versions récentes de Git pour renforcer la résistance aux collisions cryptographiques. Pour un usage courant, SHA-1 reste parfaitement sûr.

Architecture interne de Git

2.1 Les quatre types d'objets

Tout dans Git repose sur quatre types d'objets stockés dans le répertoire `.git/objects`. Comprendre ces objets, c'est comprendre pourquoi Git se comporte comme il le fait.

Les quatre objets fondamentaux de Git

Objet	Rôle	Contenu
blob	Contenu d'un fichier	Données brutes du fichier (sans nom)
tree	Répertoire	Liste de blobs et trees avec leurs noms et permissions
commit	Instantané + métadonnées	tree racine, parent(s), auteur, date, message
tag	Étiquette signée	Référence à un commit avec métadonnées optionnelles

Un point fondamental souvent mal compris : Git ne stocke PAS des différences (diffs). Il stocke des instantanés complets. Chaque commit référence un tree qui représente l'état complet du projet à cet instant. Ce choix de conception explique pourquoi les opérations comme checkout ou log sont si rapides : Git n'a pas besoin de 'rejouer' une série de patches pour reconstruire un état.

Déduplication par contenu

Si deux fichiers ont le même contenu, Git les stocke comme un seul blob. Si vous ne modifiez pas un fichier entre deux commits, le nouveau tree pointe vers le même blob que l'ancien — aucune copie. C'est ce mécanisme qui rend Git efficace en espace disque malgré le stockage d'instantanés complets.

2.2 Le DAG — Directed Acyclic Graph

L'historique Git est un graphe acyclique dirigé (DAG). Chaque commit pointe vers son ou ses parents (un commit de merge a deux parents). Les arêtes vont du passé vers le futur — 'acyclique' signifie qu'on ne peut jamais remonter d'un commit vers lui-même en suivant les pointeurs parents. Cette structure garantit que l'historique est toujours cohérent et immuable.

Visualisation du DAG

```
# Visualiser le DAG dans le terminal

git log --oneline --graph --all

# Exemple de sortie :
```

```
# * a3f92bc (HEAD -> main) feat: add push notifications
# * 7e1d034 fix: correct JWT validation
# |\
# | * 4c82a1e (feature/orders) feat: order status transitions
# | /
# * 9b3c41d chore: initial project structure
```

2.3 L'index (staging area)

L'index est une zone intermédiaire entre le répertoire de travail et le dépôt. C'est l'une des fonctionnalités les plus puissantes — et les moins comprises — de Git. L'index contient un instantané du prochain commit que vous êtes en train de construire. Vous pouvez ajouter des fichiers à l'index un par un, un fragment à la fois (`git add -p`), ou retirer des changements (`git restore --staged`). Cela vous permet de construire des commits chirurgicaux et cohérents, même si votre répertoire de travail contient des modifications en vrac.

Travailler avec l'index

```
# Ajouter un fichier entier à l'index
git add src/routes/orders.ts

# Ajouter interactivement fragment par fragment
git add -p src/routes/orders.ts

# Voir l'état de l'index vs working tree vs dernier commit
git status
git diff # working tree vs index
git diff --cached # index vs dernier commit
```

2.4 Les références

Une référence (ref) est simplement un fichier texte contenant un hash SHA-1. Les branches (`refs/heads/*`) pointent vers le dernier commit de la branche. HEAD est une référence spéciale qui pointe vers la branche courante (ou directement vers un commit en mode 'detached HEAD'). Les tags (`refs/tags/*`) sont des références permanentes vers un commit spécifique, typiquement utilisées pour marquer des versions publiées.

■ Bonne pratique

Comprendre que les branches sont de simples fichiers texte aide à démystifier Git. Créer une branche coûte quelques octets sur disque et quelques microsecondes. N'hésitez jamais à créer des branches — c'est gratuit.

Workflow quotidien

3.1 Initialiser et cloner

Deux commandes permettent de démarrer avec Git. `git init` crée un nouveau dépôt dans le répertoire courant en créant le répertoire caché `.git`. `git clone` copie intégralement un dépôt distant — historique, branches, tags — vers votre machine. Après un clone, Git configure automatiquement le remote 'origin' pointant vers l'URL source.

```
# Créer un nouveau dépôt
mkdir mon-projet && cd mon-projet
git init

# Cloner un dépôt existant
git clone https://github.com/djiguitech/social-seller.git
git clone git@github.com:djiguitech/social-seller.git # via SSH (recommandé)

# Configuration obligatoire (une seule fois par machine)
git config --global user.name "Mohamed Fomba"
git config --global user.email "fombamoh1@gmail.com"
git config --global core.editor "code --wait"
git config --global init.defaultBranch "main"
```

3.2 Le cycle add / commit

Le workflow de base de Git suit un cycle en trois états : Modified (fichier modifié dans le working tree), Staged (ajouté à l'index), Committed (enregistré dans le dépôt). Un bon commit respecte deux principes : l'atomicité (une seule unité logique de travail, ni trop petite ni trop grande) et l'auto-description (le message seul doit permettre de comprendre le changement sans lire le code).

Convention Conventional Commits

```
# Vérifier l'état avant de commiter
git status

# Ajouter des fichiers à l'index
git add src/routes/stats.ts
git add supabase/migrations/014_top_products_stats.sql

# Commiter avec un message descriptif
```



```
git commit -m "feat(stats): add top products endpoint with get_top_products() RPC"

# Convention de message (Conventional Commits) :
# feat(scope): nouvelle fonctionnalité
# fix(scope): correction de bug
# chore(scope): maintenance, refactoring sans impact fonctionnel
# migration: nouvelle migration SQL
# docs: documentation uniquement
# test: ajout ou modification de tests
```

■ Erreur fréquente

Ne jamais utiliser `git add .` de façon aveugle. Toujours inspecter `git status` et `git diff` avant d'ajouter des fichiers. Vous risquez de commiter des fichiers de configuration locaux, des clés API, ou des fichiers temporaires.

3.3 Inspecter l'historique

Git offre de nombreux outils pour naviguer dans l'historique. La commande `git log` accepte des dizaines d'options pour filtrer et formater la sortie. `git show` permet d'inspecter le contenu d'un commit. `git blame` identifie la dernière modification de chaque ligne d'un fichier — utile pour comprendre pourquoi un choix a été fait.

```
# Historique compact avec graphe
git log --oneline --graph --all

# Historique d'un fichier spécifique
git log --follow -- src/routes/orders.ts

# Rechercher dans les messages de commit
git log --grep="auth" --oneline

# Inspecter un commit précis
git show a3f92bc

# Qui a modifié cette ligne ?
git blame src/middleware/auth.ts

# Comparer deux commits ou branches
git diff main feature/promotions -- src/routes/
```

3.4 Annuler et corriger

Git offre plusieurs mécanismes pour corriger des erreurs, chacun avec des implications différentes sur l'historique. Il est crucial de comprendre la distinction entre les opérations qui réécrivent l'historique (ne

jamais faire sur des commits déjà poussés sur un remote partagé) et celles qui l'étendent (toujours safe).

Comparaison des commandes d'annulation

Commande	Effet	Réécrit l'historique ?
git commit --amend	Modifie le dernier commit (message ou contenu)	Oui
git revert	Crée un nouveau commit annulant le commit ciblé	Non
git reset --soft HEAD~1	Annule le commit, garde les changements dans l'index	Oui
git reset --hard HEAD~1	Annule le commit ET les changements (DANGEREUX)	Oui
git restore	Annule les modifications non-stagées d'un fichier	Non
git restore --staged	Retire un fichier de l'index (unstage)	Non

■■ Attention

git reset --hard est irréversible si vous n'avez pas de backup. Les modifications non-committées sont définitivement perdues. Préférez git stash pour mettre de côté des changements temporairement.

Branches — travailler en parallèle

4.1 Qu'est-ce qu'une branche ?

Une branche Git est un simple pointeur mobile vers un commit. Quand vous créez une branche, Git crée un fichier texte de 41 octets (le hash SHA-1 + saut de ligne) dans `.git/refs/heads/`. C'est tout. Lorsque vous committez sur une branche, le pointeur avance automatiquement vers le nouveau commit. HEAD est un pointeur qui indique sur quelle branche vous travaillez.

Cette légèreté des branches est l'une des différences fondamentales avec les systèmes centralisés où créer une branche impliquait souvent de copier physiquement des fichiers. En Git, la création de branche est quasi-instantanée quelle que soit la taille du projet.

```
# Créer une branche et basculer dessus
git switch -c feature/promotions

# Équivalent ancien (toujours fonctionnel)
git checkout -b feature/promotions

# Lister toutes les branches (locales + remote)
git branch -a

# Supprimer une branche fusionnée
git branch -d feature/promotions

# Forcer la suppression (non-fusionnée)
git branch -D feature/abandonned
```

4.2 Merge vs Rebase

Quand vous voulez intégrer le travail d'une branche dans une autre, vous avez deux outils principaux : merge et rebase. Ils aboutissent au même résultat final (le code est intégré) mais produisent des historiques radicalement différents.

git merge préserve l'historique exact : le moment où la branche a divergé, les commits intermédiaires, et le commit de merge final qui unit les deux lignes. L'historique est fidèle à la réalité mais peut devenir complexe à lire sur un grand projet.

git rebase rejoue les commits de votre branche sur la pointe de la branche cible. L'historique résultant est linéaire et propre, comme si vous aviez développé votre fonctionnalité directement depuis le dernier commit de main. En contrepartie, rebase réécrit les hashes des commits — ne jamais rebaser une

branche publique partagée avec d'autres développeurs.

Merge vs Rebase — workflow

```
# Merge : fusionner feature/orders dans main
git switch main
git merge feature/orders

# Crée un commit de merge si nécessaire

# Rebase : rejouer les commits de feature/ sur main
git switch feature/orders
git rebase main

# Puis fast-forward merge sur main
git switch main
git merge feature/orders # fast-forward, pas de commit de merge

# Fast-forward merge (quand main n'a pas avancé)
# Pas de commit de merge créé – historique linéaire
```

4.3 Résolution de conflits

Un conflit de merge survient quand Git ne peut pas automatiquement réconcilier deux versions d'un même fragment de fichier. Cela arrive quand deux branches ont modifié les mêmes lignes de façon incompatible. Git marque les conflits dans les fichiers avec une notation spéciale.

```
# Après git merge ou git rebase – fichier en conflit :
<<<<<<< HEAD (votre branche courante)
return res.status(200).json({ status: 'ok', version: '2.0' });
=====
return res.status(200).json({ status: 'ok', uptime: process.uptime() });
>>>>>> feature/monitoring (branche mergée)

# Résolution manuelle : éditer le fichier pour garder ce qu'on veut
return res.status(200).json({
  status: 'ok',
  version: '2.0',
  uptime: process.uptime()
});

# Puis marquer comme résolu et continuer
git add src/routes/health.ts
git merge --continue # ou git rebase --continue
```

Collaboration avec GitHub

5.1 Remote, push, pull, fetch

Un remote est un dépôt Git hébergé ailleurs — sur GitHub, GitLab, ou un serveur privé. La commande `git remote -v` liste les remotes configurés. Par convention, le remote principal s'appelle 'origin'. `git push` envoie vos commits locaux vers le remote. `git fetch` télécharge les nouvelles données du remote sans modifier votre working tree. `git pull` est un raccourci qui fait fetch puis merge (ou rebase selon la configuration).

```
# Pousser votre branche vers GitHub
git push origin feature/promotions

# Pousser et créer le tracking automatiquement
git push -u origin feature/promotions

# Récupérer les changements sans merger
git fetch origin

# Mettre à jour main depuis origin
git switch main
git pull origin main

# Voir l'état du remote
git remote -v
git remote show origin
```

5.2 Pull Requests et code review

Une Pull Request (PR) sur GitHub — appelée Merge Request sur GitLab — est une demande de fusionner une branche dans une autre. C'est le mécanisme central de collaboration : vous poussez votre branche, ouvrez une PR vers main, un ou plusieurs coéquipiers relisent le code, laissent des commentaires, et approuvent ou demandent des modifications. La PR ne peut être fusionnée qu'une fois les revues approuvées et les checks CI passés.

■ Bonne pratique

Une bonne PR est de taille raisonnable (< 400 lignes modifiées de préférence), accompagnée d'une description expliquant le contexte et les choix de design, et liée au ticket correspondant (ex: 'Closes #42'). Les grandes PRs sont difficiles à relire et génèrent des commentaires superficiels.

5.3 Gitflow et trunk-based development

Deux stratégies de branches dominent l'industrie. Gitflow (Vincent Driessen, 2010) utilise des branches longue durée (main, develop) et des branches feature/release/hotfix. Très structuré, adapté aux projets avec des cycles de release formels. Trunk-based development (TBD) favorise des branches courtes (< 1 jour) et un déploiement continu depuis main. Favorise les feature flags pour cacher les fonctionnalités incomplètes. Recommandé pour les équipes pratiquant le CI/CD.

5.4 GitHub Actions

GitHub Actions est une plateforme d'intégration et déploiement continus (CI/CD) intégrée à GitHub. Un workflow est défini dans un fichier YAML dans `.github/workflows/`. Il se déclenche sur des événements (push, pull_request, schedule) et exécute des jobs composés d'étapes.

Workflow GitHub Actions minimal

```
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with:
      node-version: '20'
      - run: npm ci
      - run: npm run build
      - run: npm test
```

■ À retenir

- ✓ Git stocke des instantanés complets identifiés par SHA-1, pas des différences
- ✓ L'index est la zone de préparation du prochain commit — utiliser `git add -p` pour des commits précis
- ✓ Une branche est un simple pointeur de 41 octets — créer des branches sans hésitation

- ✓ Merge préserve l'historique réel ; Rebase produit un historique linéaire (ne jamais rebaser du public)
- ✓ GitHub Actions automatise tests et déploiement à chaque push

—■ Exercices

Exercice 1 — Archéologie de dépôt

Sur un dépôt Git existant (votre projet ou un dépôt open source), explorez l'historique en profondeur.

- Lister les 20 derniers commits avec `git log --oneline`
- Identifier le commit qui a introduit un fichier spécifique avec `git log --follow`
- Inspecter le contenu d'un commit avec `git show`
- Utiliser `git blame` sur un fichier pour trouver qui a écrit chaque ligne

Exercice 2 — Feature branch workflow

Simulez un workflow de développement complet avec branches et PR.

- Créer une branche `feature/monprofil` depuis `main`
- Faire au moins 3 commits atomiques avec des messages Conventional Commits
- Simuler un conflit en modifiant le même fichier sur `main` et sur votre branche
- Résoudre le conflit manuellement et finaliser la fusion

Exercice 3 — GitHub Actions

Mettre en place un pipeline CI minimal sur un projet Node.js.

- Créer le fichier `.github/workflows/ci.yml`
- Le workflow doit s'exécuter sur `push` et `pull_request` vers `main`
- Configurer les étapes : `checkout`, `setup-node`, `npm ci`, `tsc --noEmit`, `npm test`
- Vérifier dans l'onglet Actions de GitHub que le workflow s'exécute correctement

Bibliographie & Ressources

Livres de référence

- Scott Chacon & Ben Straub — Pro Git (2e éd., 2014) — disponible gratuitement sur git-scm.com/book
- Jon Loeliger & Matthew McCullough — Version Control with Git (O'Reilly, 2012)

Ressources en ligne

- git-scm.com/docs — documentation officielle complète
- learngitbranching.js.org — visualisation interactive du DAG
- conventionalcommits.org — spécification Conventional Commits
- docs.github.com/actions — documentation GitHub Actions

Outils complémentaires

- lazygit — interface TUI pour Git dans le terminal
- GitLens (VS Code) — intégration Git avancée dans l'éditeur